ADR 001 — Solaris ID: Permanent Portable Identity Above Endpoints

- **Status:** Accepted
- **Date:** 2026-07-24
- **Grounding:** GPS Protocol Suite v1.0 (see `docs/GPS_PROTOCOL_NOTES.md`), esp. Constitution §6: "Identity is stable above replaceable payment endpoints." The Solaris ID is "the durable identity and permission graph above wallet endpoints" — it resolves replaceable `payment_endpoints`, and "address rotation must not erase contribution history." "Never make an address the identity."

Context

Today the app's canonical identity is the `users` row (UUID + email/password). Email is a login credential and PII — it cannot be the permanent, portable, shareable identity that GPS receipts, agent authority grants, AI execution receipts and vault exports should reference. The GPS protocol expects a stable identity (`gps:identity:` prefixed in the suite) that survives changes of email, wallet, DID or nostr key.

Decision

1. Permanent Solaris Subject ID

Every user gets exactly one **Solaris Subject ID**:

- Format: `sol_` + 32 lowercase hex chars (128 bits of randomness), e.g. `sol_1a2b3c4d5e6f7a8b9c0d1e2f3a4b5c6d` — regex `^sol_[0-9a-f]{32}$`, 36 chars total.
- **Non-PII by construction:** random; never derived from email, name, user UUID or any attribute. Contains no information about the person.
- **Permanent:** never rotated, never reused, survives credential and endpoint changes.
- Stored in `solaris_subjects` with a 1:1 `user_id` link. The `users` table remains the authentication record; the Solaris ID becomes the **canonical join key** for protocol-facing records (receipts, grants, exports).

2. Bindings, not identities

Email, DID, nostr npub, wallet addresses and (future) clinic IDs are **bindings** attached to the subject — replaceable pointers, never the identity itself.

`solaris_identity_bindings`:

- `binding_type` enum (CHECK): `email`, `did`, `nostr`, `wallet`, `clinic`.
- Implemented today: `email` (login credential), `did`, `nostr`, `wallet` (backfilled where present).
- Schema-ready, surfaced as honest "coming soon": `clinic` (and additional `wallet` / `did` flows with real verification).
- `status` enum (CHECK): `active`, `pending`, `revoked` — with `verified_at` / `revoked_at` timestamps. Lifecycle: created (`pending` or `active`) → verified → revoked. Revoking a binding never touches the subject or its history.

- **PII stance:** `binding_value` stores only public identifiers (DID strings, npubs, wallet addresses). For `email`, `binding_value` is `NULL` — only `binding_hash` (SHA-256 of the lowercased address) is stored, enough to prove/match the binding without duplicating PII outside the `users` table.
- Uniqueness: `(subject_id, binding_type, binding_hash)`.

3. GPS end-address lives on the subject

Per Constitution §6 and Comms Guide §6 (“your Solaris ID can point to a replaceable Lightning address or wallet connection”), the subject row carries the GPS end-address configuration:

- `gps_end_address` / `gps_end_address_type` — defaults `solaris_default` (Solaris-managed recipients), the state described in the notes: “defaults to Solaris-managed recipients until the user sets their own endpoint.”
- Users may set a Lightning-address-shaped text value (`name@domain`, type `lightning_address`). This is **configuration only — simulated, no real payments** are made in this app; the UI says so. Real Lightning/NWC/Spark adapters remain future work.
- `backend/src/lib/gps/protocol-config.js` `IDENTITY` remains the system-wide default/fallback; the subject row is the per-user source of truth.

4. Receipts and grants stamp the subject

Additive, nullable `subject_id` columns (FK → `solaris_subjects.subject_id`) on:

- `gps_allocation_receipts` (patient’s subject, via `gps_transactions.patient_id`),
- `ai_execution_receipts` (`user_id` → subject),
- `agent_capability_grants` (`owner_subject_id`, owner → subject).

Backfilled where mappable; stamped best-effort on all new records. Historical receipt evidence blobs are append-only and are **not** rewritten (hash-stable); the column is a join key, not part of the signed evidence.

5. Backfill strategy (migration 023, additive only)

1. Create `solaris_subjects` + `solaris_identity_bindings`.
2. Insert exactly one subject per existing user (`'sol_' || replace(gen_random_uuid()::text,'-','')`); idempotent (`ON CONFLICT (user_id) DO NOTHING`). Never duplicates user records.
3. Backfill bindings: `email` (hash-only) for all users; `did` / `nostr` where set on `users`; `wallet` from `wallet_addresses`.
4. Add + backfill the nullable `subject_id` columns of §4.
5. New users get a subject lazily via `ensureSubjectForUser(userId)` in the identity module (idempotent insert), invoked from identity endpoints and receipt stamping — no auth-flow change.

6. Agent boundary

LUCA never holds root identity keys. The agent operates only under scoped, revocable `agent_capability_grants` tied to the owner’s subject; it can neither create, rotate nor revoke subjects or bindings. (Matches the FAQ truth: “Does AI control my wallet? No.”)

Consequences

- Users get a durable, portable, plain-language identity anchor (“Your Solaris ID”) that is safe to show, copy and export — with technical detail behind advanced disclosures.

- Protocol records join on a non-PII key; email can change (or be removed from exports) without breaking contribution history.
- Vault export gains a `subject + bindings` section (PII-safe), with roundtrip test coverage.
- Future binding types (clinic ID, verified DID/npub/wallet flows) are additive rows, not schema changes.

Alternatives considered

- **Use `users.id` as the public identity** — rejected: leaks an internal DB key, no protocol prefix, cannot decouple auth record from identity graph.
- **Derive the ID from email/npub** — rejected: derivation makes the “identity” dependent on a replaceable, PII-bearing binding — the exact anti-pattern Constitution §6 forbids.
- **DB trigger for subject creation** — rejected in favor of lazy application-level `ensureSubjectForUser` (simpler, testable, keeps migrations declarative).